

Praktyka Programowania

Laboratorium 6

Podstawy konteneryzacji z użyciem Docker.

Wdrażanie skonteneryzowanej aplikacji na środowisko chmurowe i jego konfiguracja

prowadzący: mgr inż. Michał Jaroszczuk, dr inż. Konrad Kłuwak

1 Cel laboratorium

Celem laboratorium jest zapoznanie się z podstawami metodyki DevOps. W ramach zajęć studenci będą musieli zapoznać się z narzędziem do konteneryzacji **Docker**, wdrożyć skonteneryzowaną aplikację na środowisko chmurowe oraz zastosować narzędzie do zarządzania aplikacjami kontenerowymi **Kubernetes**. Praca będzie oceniana na bieżąco podczas zajęć po ukończeniu każdego z etapów. Po zajęciach wymagane będzie napisanie sprawozdania z wykonanej pracy i wysłanie go mailem do prowadzącego. Sprawozdanie powinno zawierać krótki opis wykonanych zadań.

2 Teoria i przydatne linki

2.1 Metodologia DevOps

Obserwując zmiany w świecie IT, związane z postępowaniem technologicznym, można zauważyć potrzebę szybkiego i łatwego wypuszczania zmian w oprogramowaniu oraz zwinnego dostosowywania się do zmieniających się wymagań klienta. Odpowiedzią na te potrzeby są metodyki zwinnego wytwarzania oprogramowania (Agile) oraz podejście DevOps różniące się tym, że Agile kładzie większy nacisk na współpracę pomiędzy klientem, zespołem wytwórczym i kierownictwem koncentrując się na iteracyjnym rozwoju oprogramowania. DevOps natomiast koncentruje się bardziej na automatyzacji testów, integracji i dostarczaniu, łącząc ze sobą zespoły wytwórcze (Dev) i operacyjne (Ops). Oznacza to więc, że DevOps można traktować jako rozwinięcie praktyk Agile lub ich brakujący element.



Rysunek 1: Diagram komponentów wchodzących w skład DevOps

Termin DevOps zaproponował w 2009 Patrick Dubois na pierwszej konferencji z serii DevOps Days w Gandawie w Belgii. Źródeł metodyki należy szukać w dużych firmach, które zauważyły że

brak koordynacji działu rozwoju i operacji jest źródłem strat. Tradycyjny model tworzenia oprogramowania wymagał bowiem, aby twórcy kodu byli organizacyjnie i funkcjonalnie oddzieleni od tych, którzy ten kod wdrażają i wspierają. Programiści i specjaliści działów IT i operacji mieli oddzielne, a czasami nawet konkurujące ze sobą cele, oddzielne kierownictwo czy też oddzielne wskaźniki wydajności. W efekcie zespoły były zamknięte w organizacyjnych silosach i skupione na własnych celach i klientach. Poszukując rozwiązania, które przełamać impas, obie społeczności zaczęły ze sobą rozmawiać. Sama metodyka składa się z kilku podstawowych praktyk, odróżniających je od metodologii Waterfall, gdzie cały proces tworzenia nowej funkcjonalności jest szczegółowo zaplanowany na samym początku i wykonywany w określonej kolejności. W porównaniu z modelem kaskadowym można wyróżnić następujące zalety DevOps:

- **Krótszy czas dostarczania** - W DevOps cykl dostarczania oprogramowania jest znacznie krótszy niż w Waterfall, ponieważ automatyzacja przyspiesza procesy wydawnicze,
- **Zmniejszone ryzyko** - DevOps redukuje ryzyko błędów poprzez automatyzację i ciągłe testowanie. W Waterfall, błędy mogą być wykrywane dopiero na późniejszych etapach, co może prowadzić do kosztownych poprawek. W przypadku zmian wymagań w trakcie trwania projektu, ich modyfikacja również będzie łatwiejsza w DevOps,
- **Lepsza współpraca** - DevOps promuje współpracę między zespołami, co pomaga w lepszym zrozumieniu wymagań i zwiększa jakość oprogramowania. W Waterfall, zespoły pracują sekwencyjnie, co może prowadzić do silosów informacyjnych,
- **Elastyczność** - DevOps pozwala na szybkie wprowadzanie zmian i dostosowywanie się do nowych wymagań. W Waterfall, proces jest sztywny, a zmiany mogą być trudne do wprowadzenia po rozpoczęciu projektu.

Warto jednak podkreślić, że dla pewnych specyficznych organizacji, nie będzie on dobrą metodyką pracy i znacznie lepszym podejściem może okazać się Waterfall (np. przy wdrażaniu zmian prawnych lub systemach, które nie mogą być aktualizowane na bieżąco).

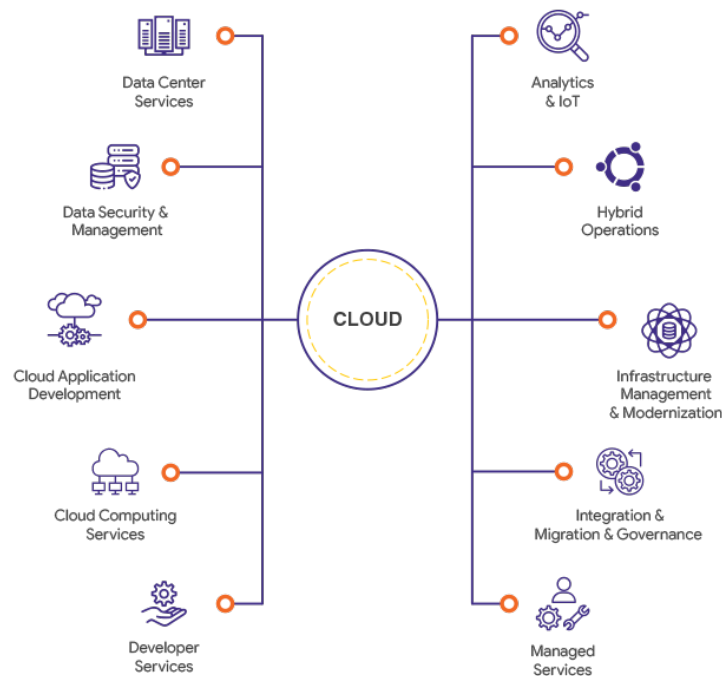
Kluczowe elementy DevOps to między innymi:

- **Kultura współpracy** - DevOps zrywa z barierami pomiędzy zespołami deweloperskimi, QA i operacyjnymi,
- **Automatyzacja** - Zamiast polegać na manualnych, czasochłonnych i podatnych na błędy operacjach, organizacje implementujące DevOps korzystają z narzędzi i technologii, które automatyzują te procesy,
- **Ciągła integracja (CI) i Ciągłe wdrażanie (CD)** - DevOps wprowadza ciągły przepływ pracy, gdzie kompilacja, testowanie i wdrażanie odbywają się regularnie i automatycznie,
- **Feedback i nieustanne ulepszanie** - Procesy DevOps zachęcają do zbierania ciągłego feedbacku od użytkowników, co pozwala na szybkie wprowadzanie niezbędnych zmian,
- **Elastyczność i skalowalność** - Współczesne aplikacje muszą radzić sobie z rosnącym i zmieniającym się obciążeniem. Kluczową rolę w sprostaniu tym wymaganiom odgrywają technologie konteneryzacji i orkiestracji. Zespoły mogą ponadto wprowadzać poprawki w czasie rzeczywistym,
- **Monitorowanie i logowanie** - Efektywne monitorowanie i logowanie są kluczowymi składnikami w każdej strategii DevOps, umożliwiając organizacjom lepsze zrozumienie działania ich aplikacji i infrastruktury w czasie rzeczywistym.

2.2 Technologie chmurowe

2.2.1 Ogólnie o rozwiązaniach chmurowych

W ostatnim czasie dużą popularność zyskują rozwiązania chmurowe, które bardzo często wykorzystywane są przy DevOpsie. Chmura najprościej mówiąc, jest modelem dostarczania usług obliczeniowych przez Internet. Zamiast korzystać z lokalnych zasobów, takich jak serwery czy magazyny danych, użytkownicy mogą uzyskać dostęp do zasobów zdalnych, które są utrzymywane przez dostawcę chmury. Dostawca zarządza infrastrukturą, zapewniając jej skalowalność, elastyczność i dostępność. Chmurę można porównać do leasingu samochodu. Zamiast kupować własny samochód, możesz zapłacić miesięczną opłatę za użytkowanie pojazdu. To pozwala skupić się na tym, co najważniejsze, czyli na użytkowaniu pojazdu, bez konieczności martwienia się o naprawy, ubezpieczenie czy finansowanie dużych kosztów początkowych. Chmura działa podobnie, umożliwiając dostęp do infrastruktury wtedy kiedy jest potrzebna, co pozwala skupić się na swojej działalności, nie martwiąc się o utrzymanie sprzętu.



Rysunek 2: Przykładowe usługi wchodzące w skład rozwiązań chmurowych

Największymi dostawcami chmury są m.in. Microsoft (**Microsoft Azure**), Amazon (**Amazon WebServices - AWS**) oraz Google (**Google Cloud Platform**). Oczywiście istnieje wielu innych dostawców rozwiązań chmurowych a ich wybór powinien być oparty o takie kryteria jak rodzaj świadczonych usług, lokalizacja serwerów, integracja z istniejącą infrastrukturą czy też koszty i modele płatności. Zalety chmury to między innymi:

- **Elastyczność** - Chmura pozwala na skalowanie zasobów w górę i w dół w zależności od potrzeb. Można szybko dodawać lub usuwać zasoby obliczeniowe, magazyny danych czy sieci,
- **Oszczędności** - Model płatności za użycie oznacza, że płacone jest tylko za zasoby, które faktycznie zostały użyte. To eliminuje potrzebę inwestowania w drogi sprzęt i utrzymania infrastruktury,

- **Mobilność** - Dzięki chmurze można uzyskać dostęp do swoich zasobów z dowolnego miejsca, o ile zapewnione jest połączenie z internetem, co sprawdzi się w zespołach pracujących zdalnie,
- **Wydajność i Skalowalność** - Chmura oferuje ogromną moc obliczeniową, która pozwala na wykonywanie złożonych operacji, takich jak analiza danych czy sztuczna inteligencja,

Należy pamiętać, że chmura pomimo posiadania wielu wspomnianych zalet, posiada również pewne wady, które muszą być wzięte pod uwagę przy rezygnacji z rozwiązań lokalnych. Ważnym wnioskiem jest to, iż przejście na rozwiązanie chmurowe może nie być odpowiednie dla każdej organizacji! Przykładowe wady chmury:

- **Bezpieczeństwo** - Przechowywanie danych w chmurze może budzić obawy dotyczące bezpieczeństwa. Istnieje ryzyko naruszenia danych czy ataków hakerskich,
- **Zależność od Dostawcy** - Korzystanie z chmury oznacza poleganie na usługach zewnętrznego dostawcy. Może to prowadzić do zależności i ograniczonej kontroli nad infrastrukturą,
- **Opóźnienia Sieciowe** - W porównaniu do infrastruktury lokalnej (on-premise), chmura może cierpieć z powodu opóźnień spowodowanych przesyłaniem danych przez internet,
- **Zgodność i Regulacje** - Przechowywanie danych w chmurze wymaga przestrzegania różnych przepisów i standardów zgodności, które mogą się różnić w zależności od lokalizacji.

3 Ogólny opis zadania i kryteria oceniania

Zadanie do wykonania na zajęciach będzie składało się z 3 części. Każda z nich podnosi ocenę z laboratorium o 1 stopień w górę:

- wykonanie zadania "Konteneryzacja aplikacji" z użyciem `Docker` i `Docker Compose`,
- wdrożenie aplikacji do bezpłatnej chmury,
- wdrożenie lokalnego klastra z użyciem K8S.

Na koniec zajęć, należy przesłać do prowadzącego krótkie sprawozdanie jako formę dokumentacji, zawierające opis oraz zrzut ekranu potwierdzający wykonanie każdego z wymienionych etapów.

4 Konteneryzacja aplikacji

4.1 Czym jest konteneryzacja?

Podczas pracy z oprogramowaniem często można spotkać się z sytuacją, w której aplikacja działa na jednym środowisku uruchomieniowym, a na innym nie, na przykład z powodu niezgodności wersji czy drobnych różnic pomiędzy systemami. Konteneryzacja rozwiązuje ten problem poprzez tworzenie obrazów zawierających wszystkie potrzebne pliki do uruchomienia aplikacji (kod aplikacji, biblioteki, zależności). Uruchomiony obraz nazywany jest kontenerem. Kontenery od maszyn wirtualnych różni fakt, iż posiadają one wyłącznie pliki potrzebne do uruchomienia obrazu aplikacji a nie cały system operacyjny i komunikują się bezpośrednio z jądrem systemu operacyjnego, na którym są uruchomione.

4.2 Uruchomienie prostego kontenera

4.2.1 Instalacja oprogramowania Docker Desktop

Zainstaluj na swoim komputerze oprogramowanie `Docker Desktop`. W celu utworzenia konta możesz wykorzystać studencki adres e-mail.

4.2.2 Przygotowanie plików

1. Stwórz folder, w którym umieścisz swoją aplikację.
2. W folderze umieść dwa pliki:

- `app.py` zawierający kod aplikacji webowej służący do zliczania odwiedzin strony:

```
1 import os
2 import time
3 import redis
4 from flask import Flask
5
6 app = Flask(__name__)
7 redis_url = os.environ.get('REDIS_URL', 'redis://redis:6379')
8 cache = redis.from_url(redis_url)
9
10 def get_hit_count():
11     retries = 5
12     while True:
13         try:
14             return cache.incr('hits')
15         except redis.exceptions.ConnectionError as exc:
16             if retries == 0:
17                 raise exc
18             retries -= 1
19             time.sleep(0.5)
20
21 @app.route('/')
22 def hello():
23     count = get_hit_count()
24     return f'<h1>Cześć! Ta strona była odwiedzona {count} razy.</h1>'
25
26 if __name__ == "__main__":
27     app.run(host="0.0.0.0", port=5000, debug=True)
```

- `requirements.txt` zawierający niezbędne biblioteki i zależności:

```
1 flask
2 redis
```

3. W tym samym katalogu utwórz plik o nazwie `Dockerfile` nieposiadający żadnego rozszerzenia. Jest to plik definiujący jakie akcje powinien wykonać docker przy budowaniu obrazu aplikacji:

```
1 # 1. Obraz bazowy z Pythonem
2 FROM python:3.10-slim
3
4 # 2. Ustawienie katalogu roboczego wewnątrz kontenera
5 WORKDIR /app
6
7 # 3. Skopiowanie pliku zależności i ich instalacja
8 COPY requirements.txt requirements.txt
9 RUN pip install -r requirements.txt
10
11 # 4. Skopiowanie reszty kodu aplikacji
12 COPY . .
13
14 # 5. Określenie portu, na którym działa aplikacja
15 EXPOSE 5000
16
17 # 6. Komenda uruchamiająca aplikację
18 CMD ["python", "app.py"]
```

4.2.3 Zbudowanie obrazu i uruchomienie aplikacji

1. W stworzonym katalogu projektu uruchom terminal PowerShell.
2. Zbuduj obraz aplikacji wykonując komendę `docker build`. Podaj nazwę obrazu (np. `my-app`) oraz jego wersję (np. 1.0) we wskazanych polach.

```
docker build -t <nazwa_obrazu_aplikacji>:<wersja_obrazu> .
```

3. Sprawdź czy obraz istnieje wywołując komendę `docker images`. Utworzone obrazy możesz także obejrzeć w aplikacji Docker Desktop.
4. Uruchom kontener z aplikacją wywołując komendę `docker run` i podając w niej odpowiednie argumenty:

```
docker run -p 5000:5000 <nazwa_obrazu_aplikacji>:<wersja_obrazu>
```

5. Sprawdź czy aplikacja działa pod wskazanym przy konfiguracji obrazu adresem i portem. Przeanalizuj zwrócony błąd i zastanów się dlaczego on występuje.

4.2.4 Uruchomienie podsieci w celu połączenia z bazą

W poprzednich punktach aplikacji nie udało się uruchomić, gdyż brakowało jej połączenia z instancją bazy danych. W tym punkcie spróbujesz ręcznie utworzyć wspólną sieć i uruchomić w niej dwa kontenery, odpowiedzialne za aplikację i za bazę danych.

1. Stwórz sieć wywołując poniższą komendę, w której podaj nazwę tworzonej podsieci:

```
docker network create <nazwa_sieci>
```

2. Uruchom kontener odpowiedzialny za hostowanie bazy danych `redis`. Nie musisz tworzyć jego obrazu - jeżeli obraz nie zostanie znaleziony lokalnie to Docker spróbuje wyszukać go w zasobach online:

```
docker run -d --name redis --network <nazwa_sieci> redis:alpine
```

3. Następnie uruchom kontener aplikacji, ale tym razem podając w jakiej podsieci ma zostać uruchomiony. Oczywiście należy zmienić parametry i dostosować je do nazwy stworzonego obrazu aplikacji i jej wersji oraz nazwy sieci.

```
docker run -p 5000:5000 --network <nazwa_sieci>  
<nazwa_obrazu_aplikacji>:<wersja_obrazu>
```

4. Sprawdź czy aplikacja działa pod wskazanym przy konfiguracji obrazu adresem i portem. Tym razem powinna zliczać liczbę odwiedzin po odświeżeniu strony.
5. Uruchomiony kontener możesz zatrzymać wywołując komendę:

```
docker stop <nazwa_kontenera>
```

4.3 Wykorzystanie Docker Compose

Jak można było zobaczyć w poprzednich punktach, aplikacje z reguły składają się z więcej niż jednego elementu i wymagają uruchamiania wielu kontenerów naraz (np. bazy danych, FE, BE i serwisu API). Uruchamianie każdego z tych elementów osobno za pomocą standardowego polecenia `docker run` wymagałoby ręcznego konfigurowania sieci, podpinania wolumenów i wpisywania długich, skomplikowanych komend w terminalu. Docker Compose powstał po to, aby ten proces maksymalnie uprościć. Zamiast uruchamiać każdy kontener z osobna, w Docker Compose całą architekturę aplikacji opisuje się w jednym pliku tekstowym o nazwie `docker-compose.yml`. W pliku definiuje się:

- Usługi (Services) - jakie kontenery mają wejść w skład aplikacji,
- Porty (Ports) - na jakich portach dostępna będzie aplikacja,
- Zmienne środowiskowe (Environment) - np. connection string do bazy danych,
- Sieci (Networks) - w jakich sieciach kontenery mają komunikować się między sobą.

Docker Compose w przeciwieństwie do Kubernetesa służy do uruchamiania aplikacji na jednej maszynie, podczas gdy K8S pozwala zarządzać aplikacjami na wielu maszynach (klastrze).

4.3.1 Uruchomienie Docker Compose

1. W folderze z aplikacją stwórz nowy plik `docker-compose.yml`.
2. Umieść w nim następującą zawartość:

```
1 version: '3.8'
2
3 services:
4   # Nasza aplikacja webowa
5   web:
6     build: .
7     ports:
8       - "5000:5000"
9     volumes:
10      - ./app
11     depends_on:
12      - redis
13
14   # Kontener bazy danych Redis
15   redis:
16     image: "redis:alpine"
```

3. Uruchom cały projekt komendą:

```
docker compose up -d
```

4. Sprawdź czy aplikacja działa pod wskazanym przy konfiguracji obrazu adresem i portem. W **Docker Desktop** możesz także zobaczyć, że kontener został rzeczywiście uruchomiony.
5. Możesz wyłączyć działający kontener wywołując komendę:

```
docker compose down
```

5 Docker Hub oraz deployment aplikacji do chmury

5.1 Umieszczenie obrazu w publicznym rejestrze

Pierwszym krokiem do wykonania jest umieszczenie stworzonego obrazu aplikacji w publicznym rejestrze **Docker Hub**. Działa on jak GitHub dla kodu, ale służy do przechowywania, udostępniania i pobierania gotowych środowisk aplikacji oraz systemów, które można uruchomić w ułamku sekund.

1. Zaloguj się w terminalu, otwartym z poziomu katalogu plików do swojego konta Docker, wywołując komendę `docker login`.
2. Oznacz swój obraz nazwą użytkownika wywołując komendę poniżej i wpisując do niej swój login oraz nazwę i wersję utworzonego wcześniej obrazu:

```
docker tag <nazwa_obrazu_aplikacji>:<wersja_obrazu>
<login>/<nazwa_obrazu_aplikacji>:<wersja_obrazu>
```

3. Wypchnij zmiany wywołując komendę:

```
docker push <login>/<nazwa_obrazu_aplikacji>:<wersja_obrazu>
```

4. Możesz sprawdzić czy obraz rzeczywiście został wypchnięty do chmury logując się do serwisu <https://hub.docker.com/>.

5.2 Wdrożenie obrazu na chmurę Render

Rysunek 3: Tworzenie nowego webserwisu w Render

1. Stwórz nowe repozytorium w serwisie GitHub i wrzuć tam kod aplikacji (pliki `app.py`, `requirements.txt`, `Dockerfile`).
2. Zaloguj się do chmury `render.com` z użyciem swojego konta GitHub.

3. Kliknij **New** → **Web Service** i wybierz stworzone w poprzednich punktach repozytorium z GitHuba. Możesz to zrobić m.in. poprzez podanie adresu do publicznego repo. Widoczność repozytoriów w Renderze możesz skonfigurować z poziomu Github wchodząc w **Settings** → **Applications** → **Render Configure**.
4. Ustaw następujące parametry:
 - Nazwę aplikacji w polu **Name**,
 - **Docker** w polu **Language**,
 - Darmowy plan (**Free**) w polu **Instance Type**.
5. Kliknij **Deploy Web Service**.

5.3 Dodanie instancji bazy danych

1. W panelu Render kliknij **New** → **Key Value**
2. Nadaj nazwę bazy, wybierz darmowy plan i kliknij **Create Key Value Instance**
UWAGA! W darmowej wersji możesz mieć tylko jedną instancję **Key Value**,
3. Skopiuj connection string do bazy z pola **Internal Key Value URL** w celu późniejszego użycia,
4. Wejdź w poprzednio utworzony Web Service i w zakładce **Environment** dodaj zmienną środowiskową klikając **Add Environment Variable**
 - **Key:** REDIS_URL
 - **Value:** <skopiowany_URL>
5. Zapisz zmiany i wykonaj ponowny deployment aplikacji,
6. Sprawdź działanie aplikacji pod podanym adresem.

6 Lokalny klaster z użyciem Kubernetes

6.1 Czym jest Kubernetes?



Rysunek 4: Logo usługi Kubernetes

Kubernetes, często zapisywany także jako K8S, jest platformą służącą do orkiestracji (zarządzania) kontenerami w klastrze. Kubernetes służy do automatyzacji zarządzania wieloma kontenerami rozproszonymi na różnych serwerach, rozwiązując następujące problemy infrastrukturalne:

- **Self - healing** - w przypadku awarii danego kontenera jest on automatycznie zabijany, a w jego miejsce uruchamiany jest nowy. Jeśli padnie cały serwer (Node), K8S przeniesie działające tam aplikacje na inne maszyny.

- **Auto scaling** - K8S potrafi automatycznie dokładać kolejne instancje podów, w przypadku gdy wzrasta zapotrzebowanie na usługi i wyłączać je w okresie niskiego ruchu.
- **Zero-downtime deployment** - podczas aktualizacji aplikacji, pody podmieniane są pojedynczo, przez co nie dochodzi do przestojów. Dodatkowo zapewnione jest centralne logowanie na poziomie klastra oraz procedura automatycznych rollbacków.

Pierwsze wydanie platformy miało miejsce w 2015 roku, a jej nazwa pochodzi od starogreckiego słowa oznaczającego sternika. Pozostając w morskiej analogii, można sobie wyobrazić go jako zarządcę portu, który zawiaduje załadunkiem kontenerów na poszczególne statki, pilnuje czy żaden kontener nie został uszkodzony, a gdy rośnie zapotrzebowanie na transport zamawia kolejne statki.

6.2 Zadania do wykonania

6.2.1 Włączenie K8S w Docker Dekstop

Pierwszym krokiem jest upewnienie się czy w Docker Desktop włączona jest opcja używania Kubernetes. W tym celu sprawdź czy w **Settings** → **Kubernetes** pole **Enable Kubernetes** posiada wartość **TRUE**.

6.2.2 Stworzenie pliku manifestu K8S

W folderze z aplikacją utwórz plik `k8s-deployment.yaml` (nazwa pliku jest dowolna) i wklej do niego poniższą zawartość. Zwróć uwagę na ustawiane porty, ustawioną liczbę w parametrze `replicas` oraz obraz, z którego zostanie zbudowany kontener. Obraz pochodzi z chmury Docker Hub i zawiera aplikację wyświetlającą parametry poda, na którym się ona uruchomiła.

```

1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: demo-deployment
5 spec:
6   replicas: 3
7   selector:
8     matchLabels:
9       app: demo-app
10  template:
11    metadata:
12      labels:
13        app: demo-app
14    spec:
15      containers:
16      - name: demo-container
17        image: containous/whoami
18        ports:
19        - containerPort: 80
20 ---
21 apiVersion: v1
22 kind: Service
23 metadata:
24   name: demo-service
25 spec:
26   type: NodePort
27   selector:
28     app: demo-app
29   ports:
30   - port: 80
31     targetPort: 80
32     nodePort: 30005

```

6.2.3 Uruchomienie klastra Kubernetes

1. Uruchom zapisany manifest K8S, wywołując w terminalu poniższą komendę. Jeśli plik k8s został nazwany inaczej, to wpisz jego nazwę faktyczną.

```
kubectl apply -f k8s-deployment.yaml
```

2. Sprawdź czy pody uruchomiły się poprawnie (znajdują się w statusie RUNNING), wywołując komendę:

```
kubectl get pods
```

Zwróć uwagę, że liczba podów odpowiada liczbie podanej w pliku manifestu. Chcąc uzyskać więcej szczegółów wywołaj komendę z dodatkową flagą:

```
kubectl get pods -o wide
```

3. Uruchom aplikację wpisując w przeglądarkę adres <http://localhost:30005>. Zwróć uwagę, że jeśli odświeżysz stronę lub uruchomisz ją w trybie incognito, to identyfikator podu nie zawsze jest taki sam. Oznacza, to że load balancing aplikacji działa poprawnie.

6.2.4 Testy klastra Kubernetes

1. Przetestuj self-healing (samouzdrawianie) klastra. Usuń jeden z podów wywołując komendę poniżej i podając id poda. Zwróć uwagę, że na jego miejsce natychmiastowo i automatycznie zostanie stworzony nowy pod:

```
kubectl delete pod <nazwa_poda>
```

2. Jeśli chcesz wyświetlić więcej informacji na temat danego poda i jego stanu, wywołaj komendę:

```
kubectl describe pod <nazwa_poda>
```

3. W przypadku konieczności zwiększenia mocy klastra (np. przy dużym ruchu sieciowym) można zmienić liczbę podów wywołując komendę:

```
kubectl scale deployment demo-deployment --replicas=5
```